SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

Day 1: Introduction to Software Engineering

A student-facing chapter for understanding how real software is planned, built, tested, released, and improved.

> **THE BIG QUESTION**
> How does an idea such as a food-ordering app, a UPI payment, or a college event portal become a reliable product that thousands—or millions—of people can use?

In this chapter

- What software is—and why software engineering is more than writing code.
- How familiar apps solve real problems and what can go wrong when software is poorly planned.
- The software crisis: delays, budget overruns, defects, security issues, and disappointed users.
- The Software Development Life Cycle (SDLC), explained through a College Event Management System.
- How people, requirements, design, testing, deployment, and maintenance work together.

Designed for a two-hour introductory classroom session. Read the explanations, discuss the examples, and attempt the activities as you go.

How to Use This Chapter

This chapter is written like a short book rather than a trainer's checklist. Each section introduces a concept, explains it in plain language, and connects it to a situation you may already understand. Your instructor may pause at the discussion questions and ask you to explain your reasoning.

Learning outcomes

By the end of this chapter, you should be able to:

- Explain software engineering in your own words.
- Distinguish writing a small program from engineering a software product.
- Identify common causes and consequences of software project failure.
- Name the major phases of the SDLC and explain why each phase matters.
- Apply the SDLC to a College Event Management System.
- Explain why changing a requirement can affect design, code, tests, schedule, and cost.

> **A USEFUL MINDSET**
> Whenever you use an app, ask three questions: What problem is it solving? Who depends on it? What could happen if it fails?

Chapter roadmap

Part	Main idea	Real-life connection
1. Software all around us	Software is part of everyday services	UPI, maps, shopping, messaging
2. Engineering vs. coding	Reliable products need a process	College event registration
3. Software crisis	Projects fail for predictable reasons	Admission portal under pressure
4. SDLC	Work is divided into connected phases	From event idea to live portal
5. Change and teamwork	Requirements evolve and roles collaborate	Adding QR attendance
6. Review	Check understanding and apply it	Quiz and mini design challenge

1. Software Is All Around Us

Think about the first hour of your day. You may silence an alarm, check a message, read a notification, open a map, scan a QR code, pay for a coffee, or book a cab. Each action depends on software. Often, we notice software only when it stops working.

Software is a collection of programs, data, and related instructions that tell a computer or device what to do. A modern software product may also include a user interface, databases, APIs, security controls, monitoring, documentation, and processes for updates and support.

A mobile app is therefore not just the screens you tap. When you open a food delivery app, several parts may work together: the app on your phone, a server, a restaurant system, a payment service, a map service, a database, and a delivery partner's app.

Example: What happens when you order food?

1. **You choose food.** The app displays menus and prices, often fetched from a server.
2. **You place an order.** The system checks details such as selected items, delivery address, and availability.
3. **The order is stored.** A database keeps a record so the order does not exist only on your phone.
4. **The restaurant receives it.** A restaurant-facing system shows the order and its status.
5. **Payment is processed.** A payment provider may confirm success, failure, or a pending result.
6. **The delivery is tracked.** Location updates and status changes are shown to the customer.
7. **The service learns and improves.** Teams monitor errors, investigate complaints, and release fixes.

> **NOTICE THE HIDDEN COMPLEXITY**
> A screen can look simple while the system behind it is complex. A button labelled “Pay” may trigger validation, network requests, security checks, transaction records, payment-provider communication, and recovery logic for failures.

Real-life example: UPI payment

Imagine you scan a merchant's QR code and pay ■250. The app must identify the recipient, request authorization, communicate with payment systems, show the correct status, and avoid misleading you if the network becomes slow. If the app shows “failed” but the amount is actually debited, the user may pay twice or lose trust. That is why payment software must be designed for correctness, security, traceability, and recovery—not merely for a good-looking screen.

Real-life example: navigation app

A navigation app combines location data, maps, traffic information, route calculation, and user interaction. If it gives an incorrect turn, the result may be more than an inconvenience: a driver could miss an appointment or enter an unsafe road. Data quality, timely updates, usability, and testing all matter.

Discussion: Choose one app you used today. Identify the problem it solves, the people who depend on it, and one consequence if it stops working.

2. What Is Software Engineering?

Software engineering is a systematic, disciplined, and measurable approach to developing, operating, testing, and maintaining software. The key idea is not that every project must be complicated. The key idea is that software should be created in a way that makes its quality, cost, schedule, and behaviour manageable.

Writing code is an important part of software development, but software engineering includes much more. Before code is written, a team needs to understand the problem. During development, it needs to make design decisions, coordinate work, review changes, and test the result. After release, it must monitor the system, fix defects, handle new needs, and protect users.

Writing code only	Software engineering
Starts by implementing an idea	Clarifies the problem and expected outcome
May rely on assumptions	Records and validates requirements
Focuses on individual functions	Considers the complete product and its users
Tests only when convenient	Plans testing throughout development
May have no clear release or recovery plan	Plans deployment, monitoring, support, and rollback
Can be difficult for others to change	Uses understandable design, documentation, and version control

An analogy: building a house

A person may know how to lay bricks, just as a programmer may know how to write code. But a safe house also needs a plan, suitable materials, structural calculations, plumbing, electrical work, inspections, and maintenance. A construction team coordinates specialists and checks that the building meets requirements. Software engineering follows a similar principle: code is essential, but it is only one part of a dependable result.

A college event portal makes this difference clear. A student may quickly create a registration form. But a production-ready system must also consider duplicate registrations, event capacity, data privacy, admin access, confirmation messages, cancellation, reports, accessibility, and what happens when the server is unavailable.

Quality is more than “it works on my laptop”

Software quality has several dimensions. A system may perform the right calculation but still be poor software if it is too slow, insecure, confusing, unreliable, or impossible to maintain.

Quality attribute	Meaning	Example question
Correctness	Does it produce the expected result?	Is the fee calculated correctly?
Reliability	Does it keep working consistently?	Can students register during peak traffic?
Usability	Can the intended user use it easily?	Can a first-time user find registration?
Security	Does it protect data and actions?	Can a student view another student's private details?
Performance	Does it respond fast enough?	Does the event page load quickly at launch?
Maintainability	Can the team safely change it?	Can a payment method be added without breaking existing flows?
Scalability	Can it handle growth?	Can it support 2,000 users instead of 200?

KEY TAKEAWAY

> A software product is successful when it solves the right problem for its users and continues to do so reliably—not merely when the code compiles.

3. The Software Crisis: Why Projects Go Wrong

As software systems became larger and more important, teams discovered that building them was often harder than expected. Projects could be late, exceed their budgets, fail to meet user needs, contain serious defects, or become so difficult to maintain that even small changes were risky. These recurring difficulties are commonly discussed under the term **software crisis**.

The phrase does not mean that all software projects fail. It describes the gap between the growing complexity of software and a team's ability to deliver dependable results without disciplined methods.

Case study: The college admission portal

A college decides to launch an online admission portal before the next admission cycle. The team is told, “Make a website where students can apply.” The deadline is fixed, but the details are unclear.

During development, several problems appear:

- The team assumes students only need to submit a form, but the college also needs document verification, fee payment, merit lists, and status tracking.
- Different departments give conflicting instructions about required documents.
- The system is tested with 20 applications, but thousands of students submit forms on the final day.
- A payment timeout creates uncertainty about whether a fee was paid.
- The portal is released without a clear support process, so students do not know where to report problems.

The result may be delayed admissions, frustrated students, extra emergency work, financial loss, and reputational damage. Notice that the cause is not necessarily that the programmers are incapable. The project lacked a clear, coordinated approach to requirements, design, capacity, testing, and support.

Common causes of software project problems

Cause	What it looks like	A better engineering response
Unclear requirements	“Make it user-friendly” means different things to different people	Ask questions, document examples, validate with users
Uncontrolled changes	New features are added without revisiting time or cost	Assess impact and agree on priority, schedule, and scope
Weak planning	Work is underestimated and dependencies are missed	Break work into tasks, estimate, schedule, and track
Insufficient testing	Only the happy path is checked	Test normal, invalid, boundary, failure, and recovery cases
Poor communication	Developers, users, and managers have different assumptions	Use reviews, clear ownership, and shared documentation
Technical complexity	Parts of the system do not integrate properly	Design interfaces, prototypes, and integration tests
No maintenance plan	Defects and security issues remain after release	Monitor, patch, support, and improve the product

Does every bug mean software engineering failed?

No. Complex software can contain defects even when teams work carefully. Engineering aims to reduce the likelihood and impact of defects, detect them early, respond transparently, and learn from them. A responsible team also plans for incidents because networks fail, external services change, and unexpected situations occur.

> **THINK LIKE AN ENGINEER**
> When a system fails, do not stop at “Who made the mistake?” Ask: What assumption was wrong? Which check was missing? How could the team detect this earlier? What would prevent recurrence?

Class discussion

Suppose a ticket-booking app sells the same seat to two people. List at least three possible causes. Consider the user interface, server, database, timing, and testing. Then suggest one prevention or detection measure for each cause.

4. The Software Development Life Cycle (SDLC)

The Software Development Life Cycle, or SDLC, is a structured way to organise the work involved in creating and maintaining software. It helps a team move from an initial idea to a product that can be used, supported, and improved.

The names and boundaries of phases can differ between organisations. A common introductory view includes planning, requirements, design, implementation, testing, deployment, and maintenance. These phases are not always strictly separate: teams may revisit earlier decisions as they learn more.

> **SDLC AT A GLANCE**
> IDEA → PLANNING → REQUIREMENTS → DESIGN → IMPLEMENTATION → TESTING → DEPLOYMENT → MAINTENANCE
> Feedback can send the team back to an earlier phase whenever new information or a defect requires it.

Running example: College Event Management System (CEMS)

Imagine your college hosts technical festivals, cultural events, workshops, and guest lectures. Today, students register through different forms and spreadsheets. Organisers struggle to remove duplicate entries, track capacity, send confirmations, and prepare attendance lists. The college proposes a single College Event Management System.

4.1 Planning: Should we build it, and what is the goal?

Planning starts by understanding the problem, expected benefits, constraints, feasibility, resources, and risks. A team asks who will use the system, what success looks like, what resources are available, and whether the work can be completed within the expected time and budget.

CEMS example: The college wants one portal for event discovery, student registration, organiser dashboards, and attendance reports. The first release will support 10 events and up to 2,000 student accounts. The team identifies a deadline before the next festival and checks whether existing college login services can be reused.

Questions to ask:

- What problem are we solving, and for whom?
- What is included in the first release—and what is not?
- What people, time, technology, and budget are available?
- What could prevent the project from succeeding?

4.2 Requirements: What must the system do?

Requirements describe what users need from the system and the conditions under which it must operate. Requirements should be specific enough that the team can design, build, and test against them.

Functional requirements describe behaviours or services. For example: a student can browse upcoming events, register for an event, cancel a registration before the deadline, and receive a confirmation.

Non-functional requirements describe qualities or constraints. For example: the event list should load within an agreed response time under a defined load; private student data must only be visible to authorised roles; and the system should be available during the registration period.

CEMS example: “The system shall prevent a student from registering more than once for the same event.” This is more testable than “Registration should work properly.”

4.3 Design: How will the system work?

Design turns requirements into a plan for implementation. It covers the structure of the system, its screens, data, components, interfaces, security decisions, and how parts communicate. Design can be represented using diagrams, models, wireframes, and written decisions.

CEMS example: The team sketches a student event page, defines a registration record in the database, separates student and organiser permissions, and decides how the system checks event capacity. The design should prevent two simultaneous requests from taking the final seat in a way that exceeds capacity.

4. The SDLC (continued)

4.4 Implementation: Build the planned solution

Implementation is the phase most people associate with software development: programmers write code and connect the components. They may use a frontend framework, a backend service, a database, APIs, automated checks, and version control. Good implementation follows agreed conventions so that multiple people can work together and future changes remain manageable.

CEMS example: Developers build event listing, registration, organiser login, and reporting features. They use version control to track changes and code reviews to catch issues before merging work.

4.5 Testing: Does the system meet the requirements?

Testing checks software behaviour and provides evidence about quality. It includes more than clicking buttons once. Testers examine expected use, invalid input, boundary values, permissions, performance, integration, and failure recovery. Testing can reveal defects, but it cannot prove that every possible defect is absent.

CEMS example: The team tests what happens when an event has 200 seats and the 201st student tries to register. It checks duplicate registration, cancellation, organiser access, confirmation messages, and two students trying to claim the final seat at almost the same time.

4.6 Deployment: Make the software available

Deployment means releasing the software into an environment where its intended users can access it. This may involve configuration, database setup, access control, backups, monitoring, training, and a plan to undo the release if something goes wrong.

CEMS example: The college first releases the portal to a small group of organisers, checks that registration and reports work, and then opens it to all students. The team prepares a support contact and monitors errors during launch.

4.7 Maintenance: Keep it useful after release

Software is rarely finished forever. Users report defects, policies change, browsers and devices evolve, security threats appear, and new features become valuable. Maintenance includes fixing defects, adapting software to its environment, improving performance, and adding carefully prioritised capabilities.

CEMS example: After the festival, organisers request QR-code attendance, waitlists, certificates, and event feedback. The team evaluates the requests, identifies risks, estimates effort, and plans suitable updates instead of adding every request immediately.

The phases and their main outputs

Phase	Main question	Typical output for CEMS
Planning	Why build it, and what is feasible?	Goal, scope, feasibility notes, initial risks
Requirements	What do users need?	Requirements list, user stories, acceptance criteria
Design	How will it be structured?	Screen sketches, data model, architecture and diagrams
Implementation	Can we build the solution?	Working features and source code
Testing	Does it behave as expected?	Test cases, defect reports, test results
Deployment	Can users use it safely?	Released application, configuration, rollout plan
Maintenance	How will it remain useful?	Fixes, updates, monitoring, support records

Why is feedback important?

Imagine testing reveals that a student can register for a full event. The team may need to revisit the requirement, design, code, and tests. Or users may discover that they need a waitlist. The SDLC should help the team respond to new information in a controlled way. It is a map for organising work, not a promise that every decision will be correct on the first attempt.

> **REMEMBER**
> Planning prevents avoidable confusion. Requirements establish what success means. Design gives the team a coherent solution. Implementation creates it. Testing checks it. Deployment releases it. Maintenance keeps it useful.

5. Requirements Change: A Real-Life Simulation

Software is built for people, and people's needs can change. A new regulation, a user complaint, an unexpected event, or a new business opportunity can introduce a requirement after work has begun. Change is not automatically bad. Uncontrolled change is the problem.

Scenario: QR attendance is requested

The CEMS team has nearly completed event registration. The event coordinator now asks: “Can we add QR-code attendance? At the entrance, volunteers should scan a student's code and mark them present.”

At first, this sounds like a small feature. But an engineer asks follow-up questions:

- Who generates the QR code: the system, the student, or the organiser?
- Can the same code be scanned multiple times? What should happen on a duplicate scan?
- Can a student show another student's code?
- What if the internet is unavailable at the venue?
- Who is allowed to view attendance reports?
- Does this affect the release date, cost, testing effort, or volunteer training?

This example shows why requirements need analysis before a feature is promised. A single request can affect user experience, data design, security, implementation, testing, and support.

A simple change-impact method

1. **Clarify the need.** What problem does QR attendance solve? What does success look like?
2. **Identify affected areas.** Screens, database, permissions, APIs, reports, and operations.
3. **Estimate the impact.** How much work, testing, and coordination are required?
4. **Compare priority.** Is this essential for the first release or can it wait?
5. **Agree on a decision.** Accept, defer, simplify, or reject the change with reasons.
6. **Update shared records.** Revise requirements, tasks, test cases, and release plans.
7. **Verify the result.** Test the new feature and ensure existing registration still works.

Why teams need different roles

A software project involves people with different responsibilities. In a small project, one person may perform several roles. In a larger project, responsibilities are often distributed.

Role	Main concern	CEMS example
User / stakeholder	The real problem and desired outcome	Student wants quick registration; coordinator needs reports
Business analyst	Clarify and document needs	Defines duplicate-registration rules
UI/UX designer	Make tasks understandable and usable	Designs event list and registration flow
Developer	Implement the solution	Builds registration and organiser features
Tester / QA engineer	Find defects and assess quality	Checks capacity limits and access permissions
Project manager / Scrum Master	Coordinate work and remove blockers	Tracks dependencies and resolves delays

Role	Main concern	CEMS example
Operations / support	Keep the service running	Monitors errors during the festival

Short role-play: The requirement meeting

In a small group, assign the roles of student, event coordinator, developer, tester, and project coordinator. Discuss the QR-attendance request. Each person should raise one concern from their role. The group should agree on the minimum version that could be delivered safely.

There is no single perfect answer. A good answer explains its assumptions, considers users and risks, and makes trade-offs visible.

6. Different Ways to Organise Development

The SDLC describes the broad activities involved in software development. A process model describes how a team organises and moves through those activities. You will study these models in more detail in upcoming sessions. For now, understand why different projects may need different approaches.

Waterfall

Waterfall organises work into phases that generally progress in sequence. It can be useful when requirements are stable, approvals are formal, and the work is well understood. A risk is that users may see a working product late, so misunderstandings can be expensive to correct.

Example: A small internal reporting tool with well-defined regulatory outputs and stable requirements may benefit from detailed upfront planning and formal review gates.

Incremental development

Incremental development delivers a product in usable portions. Instead of waiting for every feature, the team builds a core capability and then adds more. This can provide earlier value and feedback, provided the increments are planned and integrated carefully.

CEMS example: Release 1 supports event listings and registration. Release 2 adds organiser reports. Release 3 adds QR attendance and certificates.

Prototype-based development

A prototype is an early model used to explore an idea, test a design, or clarify requirements. It may be a paper sketch, clickable screen mock-up, or limited working system. A prototype helps users react to something concrete before the team commits to a complete implementation.

CEMS example: Show students a clickable mock-up of the event-registration flow and ask them to try finding and registering for a workshop.

Spiral development

The Spiral model emphasises repeated cycles that identify objectives, evaluate alternatives, address risks, and plan the next cycle. It is especially associated with projects where risk analysis is important. It requires thoughtful risk assessment and can be costly to manage.

Example: A high-risk system involving sensitive data, complex integrations, and major consequences if it fails may need explicit risk-driven iterations.

Agile development

Agile is a family of approaches that emphasise frequent delivery, collaboration, feedback, and adapting to change. Teams often work in short iterations, review what they have built, and reprioritise upcoming work. Agile does not mean “no planning” or “no documentation”; it means planning and documentation are used in ways that support learning and delivery.

CEMS example: The team delivers a working registration flow, gets feedback from organisers, improves it, and then builds the next highest-priority feature.

Approach	Best starting question	Possible limitation
Waterfall	Can we define and approve the work clearly upfront?	Late feedback may reveal expensive misunderstandings

Approach	Best starting question	Possible limitation
Incremental	What useful part can we deliver first?	Poorly planned increments can create integration problems
Prototype	What should we learn or validate before full build?	Users may mistake a rough prototype for a finished product
Spiral	Which risks must be understood and reduced first?	Risk analysis and repeated cycles can be resource-intensive
Agile	How can we deliver, learn, and adapt frequently?	Needs active collaboration and disciplined prioritisation

The goal is not to memorise labels

When choosing a process model, consider the clarity of requirements, risk, project size, regulation, user availability, delivery deadlines, team experience, and the cost of changing a decision. A model is useful when it fits the project's context; no single model is ideal for every situation.

7. Mini Design Challenge: Build the Idea Before the Code

Work in a group and imagine that the college has asked you to design a simple CEMS. You are not expected to write code. Your task is to think like a software engineering team.

Step 1: Identify the people

List at least three stakeholder groups. For example, students, event coordinators, faculty approvers, volunteers, and college administrators. For each group, write one goal or concern.

Step 2: List features

Write at least five functional requirements. Examples include viewing events, registering, cancelling before a deadline, receiving confirmation, and downloading an attendance report. Make each requirement clear enough that someone could test it.

Step 3: Add quality requirements

Write two non-functional requirements. Avoid vague statements such as “the app must be fast.” State a measurable target or condition when possible, such as “the event list should load within two seconds for 95% of requests under the agreed test load.” The exact target should be chosen with stakeholders and realistic technical constraints.

Step 4: Identify risks

List three risks. Examples: duplicate registrations, unauthorised access to student information, or a traffic spike when registration opens. For each risk, suggest a prevention or mitigation measure.

Step 5: Plan a test

Choose one requirement and write a test with four parts: starting condition, action, expected result, and evidence. Example: when an event is full, a student attempts to register; the system should reject the registration or offer an approved waitlist option; the result should be recorded in the test report.

> **CHALLENGE QUESTION**
> If the college asks for payment collection one day before launch, what would you need to investigate before promising that it can be added?

8. Knowledge Check

Try to answer each question before reading the answer key.

1. In your own words, what is software engineering?

2. Why is a registration form alone not necessarily a complete event-management system?

3. Name any three causes of software project problems.

4. Put these activities in a sensible SDLC order: testing, requirements, maintenance, design, deployment, planning, implementation.

5. Give one functional and one non-functional requirement for CEMS.

6. Why should a team test what happens when an event reaches its capacity limit?

7. A user asks for QR attendance after development has started. Why should the team assess the impact before agreeing?

8. Which approach could help validate a confusing screen before full development? Explain your choice.

9. Does Agile mean that a team should avoid planning and documentation? Explain.

10. Name one maintenance activity that may happen after release.

9. Suggested Answers and Explanations

These are sample answers, not the only acceptable wording. Strong answers explain the reasoning and use a relevant example.

1. Software engineering is a systematic and disciplined way to develop, test, release, operate, and maintain software so that it meets user needs and quality expectations.

2. A registration form may not manage event capacity, duplicates, cancellations, permissions, confirmations, attendance, reporting, or failures. A complete system considers the full workflow.

3. Examples include unclear requirements, uncontrolled changes, weak planning, insufficient testing, poor communication, technical complexity, and lack of maintenance.

4. One common sequence is planning → requirements → design → implementation → testing → deployment → maintenance. Real projects may revisit earlier phases.

5. Functional: “A student can cancel a registration before the cancellation deadline.” Non-functional: “Only authorised organisers can access attendance reports.” A measurable performance requirement is also acceptable.

6. Capacity is a boundary condition. Testing it helps ensure the system does not overbook an event and handles the final available seat correctly.

7. QR attendance can affect requirements, screens, data, permissions, devices, network handling, reports, schedule, cost, and regression testing. The team needs to understand the impact and agree on scope.

8. A prototype is useful because users can interact with an early representation and provide feedback before the team builds the complete feature.

9. No. Agile teams plan and document, but they favour useful, appropriately sized planning and documentation, frequent feedback, and adaptation instead of treating a fixed plan as unchangeable.

10. Examples include fixing a defect, updating a security dependency, improving performance, adapting to a browser change, adding a feature, or monitoring production errors.

10. Glossary

Term	Meaning
Software	Programs, data, and related instructions that enable a computer-based system to perform tasks.
Software engineering	A systematic, disciplined, measurable approach to software development and maintenance.
Stakeholder	A person or group affected by, interested in, or able to influence a system.
Requirement	A documented need, capability, quality, or constraint the system must satisfy.
Functional requirement	A behaviour or service the system must provide.
Non-functional requirement	A quality attribute or constraint, such as performance, security, or availability.
SDLC	The life cycle of activities used to plan, build, test, release, and maintain software.
Defect / bug	A flaw that may cause the software to behave differently from what is expected.
Deployment	Making a software release available in its intended environment.
Maintenance	Work performed after release to correct, adapt, improve, and support software.
Prototype	An early model used to explore a design or validate assumptions.
Version control	A system for tracking changes to files and coordinating collaborative development.

11. Homework: Observe Software Like an Engineer

Choose one application or digital service that you personally use. Examples include a messaging app, UPI app, online shopping service, travel app, college portal, or food delivery platform. Choose one you can explain clearly.

Prepare a one-page response covering the following:

1. **Problem:** What problem does the application solve?
2. **Users:** Who uses it, and what does each user want to achieve?
3. **Features:** List three things the system must allow users to do.
4. **Quality:** Identify two important qualities, such as security, reliability, usability, or performance.
5. **Failure:** Describe one realistic failure scenario and its possible impact.
6. **SDLC:** Explain one activity that would happen during requirements, design, testing, deployment, or maintenance.
7. **Improvement:** Suggest one improvement and explain how you would check whether it works.

> **EXAMPLE OPENING**
> “I chose a UPI payment app. Its main purpose is to let users transfer money digitally. Its users include customers and merchants. Two important quality attributes are security and reliability. One failure scenario is an unclear transaction status after a network timeout…”

Before the next class

Review the five process-model names introduced in this chapter: Waterfall, Incremental, Prototype, Spiral, and Agile. You do not need to memorise every detail yet. Bring one example of a project where requirements might remain stable and one example where user needs may change frequently.

Final thought

The next time an app works smoothly, remember that the experience is usually the result of many invisible decisions: understanding the problem, choosing a design, writing and reviewing code, testing unusual situations, releasing carefully, and improving the product over time. Software engineering is the discipline that helps teams make those decisions deliberately.

End of Day 1 • Introduction to Software Engineering